

Logic optimization

Logic optimization is a process of finding an equivalent representation of the specified logic circuit under one or more specified constraints. This process is a part of a logic synthesis applied in digital electronics and integrated circuit design.

Generally, the circuit is constrained to a minimum chip area meeting a predefined response delay. The goal of logic optimization of a given circuit is to obtain the smallest logic circuit that evaluates to the same values as the original one.^[1] Usually, the smaller circuit with the same function is cheaper,^[2] takes less space, consumes less power, has shorter latency, and minimizes risks of unexpected cross-talk, hazard of delayed signal processing, and other issues present at the nano-scale level of metallic structures on an integrated circuit.

In terms of Boolean algebra, the optimization of a complex Boolean expression is a process of finding a simpler one, which would upon evaluation ultimately produce the same results as the original one.

Motivation

The problem with having a complicated circuit (i.e. one with many elements, such as logic gates) is that each element takes up physical space and costs time and money to produce. Circuit minimization may be one form of logic optimization used to reduce the area of complex logic in integrated circuits.

With the advent of logic synthesis, one of the biggest challenges faced by the electronic design automation (EDA) industry was to find the most simple circuit representation of the given design description.^[nb 1] While two-level logic optimization had long existed in the form of the Quine–McCluskey algorithm, later followed by the Espresso heuristic logic minimizer, the rapidly improving chip densities, and the wide adoption of Hardware description languages for circuit description, formalized the logic optimization domain as it exists today, including Logic Friday (graphical interface), Minilog, and ESPRESSO-IISOJS (many-valued logic).^[3]

Methods

The methods of logic circuit simplifications are equally applicable to Boolean expression minimization.

Classification

Today, logic optimization is divided into various categories:

Based on circuit representation

- Two-level logic optimization
- Multi-level logic optimization

Based on circuit characteristics

- Sequential logic optimization
- Combinational logic optimization

Based on type of execution

- Graphical optimization methods
- Tabular optimization methods
- Algebraic optimization methods

Graphical methods

Graphical methods represent the required logical function by a diagram representing the logic variables and value of the function. By manipulating or inspecting a diagram, much tedious calculation may be eliminated. Graphical minimization methods for two-level logic include:

- Euler diagram (aka Eulerian circle) (1768) by Leonhard P. Euler (1707–1783)
- Venn diagram (1880) by John Venn (1834–1923)
- Karnaugh map (1953) by Maurice Karnaugh

Boolean expression minimization

The same methods of Boolean expression minimization (simplification) listed below may be applied to the circuit optimization.

For the case when the Boolean function is specified by a circuit (that is, we want to find an equivalent circuit of minimum size possible), the unbounded circuit minimization problem was long-conjectured to be Σ_2^P -complete in time complexity, a result finally proved in 2008,^[4] but there are effective heuristics such as Karnaugh maps and the Quine–McCluskey algorithm that facilitate the process.

Boolean function minimizing methods include:

- Quine–McCluskey algorithm
- Petrick's method

Optimal multi-level methods

Methods that find optimal circuit representations of Boolean functions are often referred to as *exact synthesis* in the literature. Due to the computational complexity, exact synthesis is tractable only for small Boolean functions. Recent approaches map the optimization problem to a Boolean satisfiability problem.^{[5][6]} This allows finding optimal circuit representations using a SAT solver.

Heuristic methods

A heuristic method uses established rules that solve a practical useful subset of the much larger possible set of problems. The heuristic method may not produce the theoretically optimum solution, but if useful, will provide most of the optimization desired with a minimum of effort. An example of a computer system that uses heuristic methods for logic optimization is the Espresso heuristic logic minimizer.

Two-level versus multi-level representations

While a two-level circuit representation of circuits strictly refers to the flattened view of the circuit in terms of SOPs (sum-of-products) — which is more applicable to a PLA implementation of the design — a multi-level representation is a more generic view of the circuit in terms of arbitrarily connected SOPs, POSs (product-of-sums), factored form etc. Logic optimization algorithms generally work either on the structural (SOPs, factored form) or functional representation (binary decision diagrams, algebraic decision diagrams) of the circuit. In sum-of-products (SOP) form, AND gates form the smallest unit and are stitched together using ORs, whereas in product-of-sums (POS) form it is opposite. POS form requires parentheses to group the OR terms together under AND gates, because OR has lower precedence than AND. Both SOP and POS forms translate nicely into circuit logic.

If we have two functions F_1 and F_2 :

$$F_1 = AB + AC + AD,$$

$$F_2 = A'B + A'C + A'E.$$

The above 2-level representation takes six product terms and 24 transistors in CMOS Rep.

A functionally equivalent representation in multilevel can be:

$$P = B + C.$$

$$F_1 = AP + AD.$$

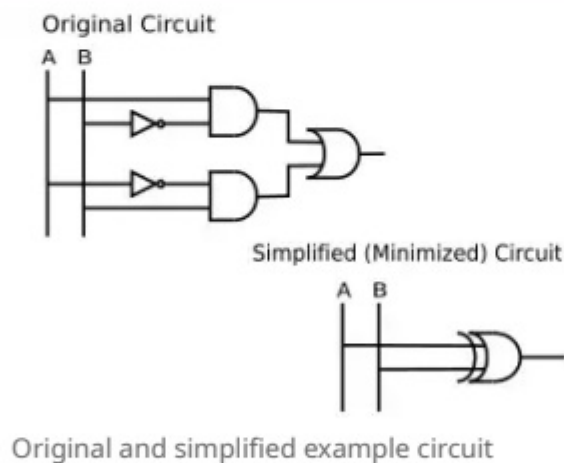
$$F_2 = A'P + A'E.$$

While the number of levels here is 3, the total number of product terms and literals reduce because of the sharing of the term $B + C$.

Similarly, we distinguish between combinational circuits and sequential circuits. Combinational circuits produce their outputs based only on the current inputs. They can be represented by Boolean relations. Some examples are priority encoders, binary decoders, multiplexers, demultiplexers.

Sequential circuits produce their output based on both current and past inputs, depending on a clock signal to distinguish the previous inputs from the current inputs. They can be represented by finite state machines. Some examples are flip-flops and counters.

Example



While there are many ways to minimize a circuit, this is an example that minimizes (or simplifies) a Boolean function. The Boolean function carried out by the circuit is directly related to the algebraic expression from which the function is implemented.^[7] Consider the circuit used to represent $(A \wedge \bar{B}) \vee (\bar{A} \wedge B)$. It is evident that two negations, two conjunctions, and a disjunction are used in this statement. This means that to build the circuit one would need two inverters, two AND gates, and an OR gate.

The circuit can simplified (minimized) by applying laws of Boolean algebra or using intuition. Since the example states that A is true when B is false and the other way around, one can conclude that this simply means $A \neq B$. In terms of logical gates, inequality simply means an XOR gate (exclusive or). Therefore, $(A \wedge \bar{B}) \vee (\bar{A} \wedge B) \iff A \neq B$. Then the two circuits shown below are equivalent, as can be checked using a truth table:

A	B		(A	$\wedge$	$\overline{B}$)	$\vee$	($\overline{A}$	$\wedge$	B)		A	$\neq$	B
F	F		F	F	T	F	T	F	F		F	F	F
F	T		F	F	F	T	T	T	T		F	T	T
T	F		T	T	T	T	F	F	F		T	T	F
T	T		T	F	F	F	F	F	T		T	F	T

See also

- Binary decision diagram (BDD)
- Don't care condition
- Prime implicant
- Circuit complexity — on estimation of the circuit complexity
- Function composition
- Function decomposition
- Gate underutilization
- Logic redundancy
- Harvard minimizing chart (Wikiversity) (Wikibooks)

Notes

1. The netlist size can be used to measure simplicity.

References

1. Maxfield, Clive "Max" (2008-01-01). "Chapter 5: "Traditional" Design Flows" (<https://www.sciencedirect.com/science/article/pii/B9780750689748000053>). In Maxfield, Clive "Max" (ed.). *FPGAs*. Instant Access. Burlington: Newnes / Elsevier Inc. pp. 75–106. doi:10.1016/B978-0-7506-8974-8.00005-3 (<https://doi.org/10.1016%2FB978-0-7506-8974-8.00005-3>). ISBN 978-0-7506-8974-8. Retrieved 2021-10-04.
2. Balasanyan, Seyran; Aghagulyan, Mane; Wuttke, Heinz-Dietrich; Henke, Karsten (2018-05-16). "Digital Electronics" (<https://ec.europa.eu/programmes/erasmus-plus/project-result-content/120e4810-0d29-4397-9ad4-b4091c2e3d19/Digital%20Electronics.pdf>) (PDF). Bachelor Embedded Systems - Year Group. Tempus. DesIRE. Archived (<https://web.archive.org/web/20211004200546/https://ec.europa.eu/programmes/erasmus-plus/project-result-content/120e4810-0d29-4397-9ad4-b4091c2e3d19/Digital%20Electronics.pdf>) (PDF) from the original on 2021-10-04. Retrieved 2021-10-04. (101 pages)
3. Theobald, M.; Nowick, S. M. (November 1998). "Fast heuristic and exact algorithms for two-level hazard-free logic minimization" (<https://academiccommons.columbia.edu/doi/10.7916/D8N58V>

- 58/download). *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*. **17** (11): 1130–1147. doi:10.1109/43.736186 (<https://doi.org/10.1109%2F43.736186>).
4. Buchfuhrer, David; Umans, Christopher (January 2011). "The complexity of Boolean formula minimization" (<http://users.cms.caltech.edu/~umans/papers/BU07.pdf>) (PDF). *Journal of Computer and System Sciences*. **77** (1). Computer Science Department, California Institute of Technology, Pasadena, California, USA: Elsevier Inc.: 142–153. doi:10.1016/j.jcss.2010.06.011 (<https://doi.org/10.1016%2Fj.jcss.2010.06.011>). This is an extended version of the conference paper: Buchfuhrer, David; Umans, Christopher (2008). "The Complexity of Boolean Formula Minimization". *Proceedings of Automata, Languages and Programming, 35th International Colloquium (ICALP)* (<http://users.cms.caltech.edu/~umans/papers/BU07.pdf>) (PDF). Lecture Notes in Computer Science. Vol. 5125. Berlin / Heidelberg, Germany: Springer-Verlag. pp. 24–35. doi:10.1007/978-3-540-70575-8_3 (https://doi.org/10.1007%2F978-3-540-70575-8_3). ISBN 978-3-540-70574-1. Archived (<https://web.archive.org/web/20180114141842/http://users.cms.caltech.edu/~umans/papers/BU07.pdf>) (PDF) from the original on 2018-01-14. Retrieved 2018-01-14.
 5. Haaswijk, Winston. "SAT-Based Exact Synthesis: Encodings, Topology Families, and Parallelism" (<https://si2.epfl.ch/~demichel/publications/archive/2020/winston-exact.pdf>) (PDF). EPFL. Retrieved 2022-12-07.
 6. Haaswijk, Winston. "SAT-Based Exact Synthesis for Multi-Level Logic Networks" (<https://si2.epfl.ch/~demichel/graduates/theses/winston.pdf>) (PDF). EPFL. Retrieved 2022-12-07.
 7. Mano, M. Morris; Kime, Charles R. (2014). *Logic and Computer Design Fundamentals* (4th new international ed.). Pearson Education Limited. p. 54. ISBN 978-1-292-02468-4.

Further reading

- Lind, Larry Frederick; Nelson, John Christopher Cunliffe (1977). *Analysis and Design of Sequential Digital Systems* (<https://archive.org/details/AnalysisDesignOfSequentialDigitalSystems/>). Macmillan Press. ISBN 0-33319266-4. (146 pages)
- De Micheli, Giovanni (1994). *Synthesis and Optimization of Digital Circuits*. McGraw-Hill. ISBN 0-07-016333-2. (NB. Chapters 7–9 cover combinatorial two-level, combinatorial multi-level, and respectively sequential circuit optimization.)
- Hachtel, Gary D.; Somenzi, Fabio (2006) [1996]. *Logic Synthesis and Verification Algorithms*. Springer Science & Business Media. ISBN 978-0-387-31005-3.
- Kohavi, Zvi; Jha, Niraj K. (2009). "4–6". *Switching and Finite Automata Theory* (3rd ed.). Cambridge University Press. ISBN 978-0-521-85748-2.
- Rutenbar, Rob A. *Multi-level minimization, Part I: Models & Methods* (<https://www.ece.cmu.edu/~ee760/760docs/lec07.pdf>) (PDF) (lecture slides). Carnegie Mellon University (CMU). Lecture 7. Archived (<https://web.archive.org/web/20180115125725/https://www.ece.cmu.edu/~ee760/760docs/lec07.pdf>) (PDF) from the original on 2018-01-15. Retrieved 2018-01-15; Rutenbar, Rob A. *Multi-level minimization, Part II: Cube/CoKernel Extract* (<https://www.ece.cmu.edu/~ee760/760docs/lec08.pdf>) (PDF) (lecture slides). Carnegie Mellon University (CMU). Lecture 8. Archived (<https://web.archive.org/web/20180115125733/https://www.ece.cmu.edu/~ee760/760docs/lec08.pdf>) (PDF) from the original on 2018-01-15. Retrieved 2018-01-15.

